

Object-Oriented Programming (OOP) Principles

Track: Data Science | Level 1 - Day 13 | Tools: Python 3, Jupyter Notebook

Level 1 - Day 13: Object-Oriented Programming (OOP) Principles

Track: Data Science

Task: Task 13 - Core OOP Architecture for Data Science

Objective: Explore core Object-Oriented Programming (OOP) paradigms: building robust class architectures, modular data pipelines, state management, and separation of concerns.

1. Quick Reference & Core Concepts

OOP Pillar	Core Idea	Machine Learning Pipeline Role
Abstraction	Hiding internal complexity behind clean interfaces	model.fit() hides optimizer updates
Encapsulation	Bundling attributes & methods while controlling access	Internal hyperparameter and weights protection
Inheritance	Deriving child classes from base parents	Custom regressors inheriting from BaseEstimator
Polymorphism	Common interface across different implementations	Unified .predict() across LinearRegression and Trees

2. Practical Code Examples

In [code]

```
# Example 1: Designing an Abstract Machine Learning Preprocessor
class BasePreprocessor:
    def __init__(self, feature_name):
        self.feature_name = feature_name

    def process(self, values):
        raise NotImplementedError("Subclasses must implement process()")

class TextCleaner(BasePreprocessor):
    def process(self, values):
        return [v.strip().lower() for v in values]

cleaner = TextCleaner("user_comments")
print(f"Ex 1 - Cleaned Text: {cleaner.process([' Hello ', 'WORLD! '])}")
```

In [code]

```
# Example 2: Data Pipeline State Management
class PipelineTracker:
    def __init__(self, step_name):
        self.step_name = step_name
        self.logs = []

    def log(self, message):
        self.logs.append(f"[{self.step_name}] {message}")

    def get_summary(self):
        return "
".join(self.logs)

tracker = PipelineTracker("DataIngestion")
tracker.log("Read 500 rows from CSV")
tracker.log("Filtered 24 corrupted rows")
print(f"Ex 2 - Tracker Summary:
{tracker.get_summary()}")
```

3. Applied Data Science Pipeline

Applied Data Pipeline: Modular Data Pipeline with Evaluator Objects

In [code]

```
class ModelEvaluator:
    def __init__(self, model_name):
        self.model_name = model_name

    def compute_mae(self, y_true, y_pred):
        return sum(abs(a - b) for a, b in zip(y_true, y_pred)) / len(y_true)

    def compute_mse(self, y_true, y_pred):
        return sum((a - b) ** 2 for a, b in zip(y_true, y_pred)) / len(y_true)

evaluator = ModelEvaluator("RandomForestRegressor")
actuals = [100, 150, 200, 250]
predictions = [95, 155, 190, 260]

print(f"Model: {evaluator.model_name}")
print(f"MAE: {evaluator.compute_mae(actuals, predictions):.2f}")
print(f"MSE: {evaluator.compute_mse(actuals, predictions):.2f}")
```